

Chapter 10 : React and APIs in Modern Web Development

1. Why React Was Introduced

Traditional websites were built using HTML, CSS, and JavaScript, where:

- HTML handled structure,
- CSS handled style,
- JavaScript handled interactivity.

However, as websites became more complex and dynamic (e.g., Facebook, Gmail), managing the user interface state, performance, and reusability of code became very difficult using plain JavaScript or jQuery.



React Emerged to Solve:

- Complex UI state management,
- Component reusability,
- Performance issues caused by DOM manipulation,
- One-way data flow for predictable logic.

2. What is React?

React is an open-source JavaScript library developed by Facebook for building user interfaces, especially single-page applications (SPA).

✅ Features of React:

- **Component-Based:** UI is broken into small, reusable components.
- **Declarative:** You describe what you want the UI to look like.
- **Virtual DOM:** React uses a lightweight copy of the real DOM for faster rendering.
- **Unidirectional Data Flow:** Data flows in one direction (top-down), making the app more predictable.

🔄 Example React Component:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

3. Difference Between React and HTML

Feature	HTML	React
Structure	Static	Dynamic
Rendering	Manual DOM	Virtual DOM
Components	Not supported	Supported
Reusability	Limited	High
Data Binding	Not built-in	Supported
Learning Curve	Easy	Moderate
Performance	Slower for dynamic UIs	Optimized with Virtual DOM

4. What is an API?

API (Application Programming Interface) is a way for different software applications to communicate. In web development, an API usually means a RESTful HTTP-based interface that allows you to get or send data from/to a backend server.

GET <https://api.example.com/users>

React + API:

In React, we often use `fetch`, `axios`, or `React Query` to get data from an API and display it.

```
useEffect(() => {  
  fetch('https://api.example.com/users')  
    .then(res => res.json())  
    .then(data => setUsers(data));  
}, []);
```

5. Types of APIs

Type	Description
------	-------------

REST API	Based on HTTP verbs (GET, POST, PUT, DELETE). Most common.
GraphQL API	Allows clients to specify exactly what data they need.
SOAP API	XML-based, used in older enterprise systems.
WebSocket API	Real-time, two-way communication (e.g., chat apps).

React vs Node.js

Feature	React	Node.js
Role	Frontend Library	Backend Runtime
Environment	Browser	Server
Used For	UI rendering	Server-side logic, APIs
Language	JavaScript	JavaScript (runtime)

React is for building what the user sees. Node.js is for handling backend logic like databases, authentication, APIs.

React vs TypeScript

Feature	React	TypeScript
---------	-------	------------

Type	UI Library	Programming Language (superset of JavaScript)
Use	UI building	Type safety in large codebases
Output	JSX -> HTML	TS -> JavaScript
Benefit	UI abstraction	Compile-time error detection

🔧 **React can be used with TypeScript for better developer experience and type safety: type Props = {**

```
  name: string;
};
```

```
const Welcome: React.FC<Props> = ({ name }) => {
  return <h1>Hello, {name}</h1>;
};
```

7. Benefits of Using React with APIs

-  **Real-time data interaction with backend services.**
-  **Easier to create dynamic UIs that reflect server state.**
-  **Clean separation of UI (React) and data (API/backend).**
-  **React's component system makes it easy to consume APIs and show data in reusable views.**

the next page in VS 😏